

第2章 算法分析

建议学时:4-5 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 理解为什么"程序能跑"远远不够——**复杂度**决定程序在数据变大时能不能跑完
- 掌握 O 、 Ω 、 Θ 、 o 四种记号的准确定义与直觉,能说出它们的区别
- 能背诵增长阶排序:常数 < 对数 < 线性 < 线性对数 < 平方 < 立方 < 指数
- 会计算顺序/循环/递归代码的运行时间,能把循环翻译成求和记号 Σ
- 亲手实现**最大子序列和**问题的四种算法,体会 $O(n^3)$ 到 $O(n)$ 的天壤之别
- 能说明二分查找、快速幂、欧几里得算法为什么都是 $O(\log n)$
- 学完能独立用 C++ `chrono` (C 中对应 `clock()`) 实测验证自己的复杂度分析

💡 从 C 到 C++:本章主题如何迁移

本章是全书唯一"几乎不写新数据结构"的一章,但它是**全书的通用语言**:第 3 章起,每章都以"这个操作 $O(\text{多少})$ "作为结论。算法分析的直觉一句话:复杂度是数据规模变大时,运行时间的增长方式——不是秒数,不是毫秒,而是"规模翻倍,时间大约乘以几"。

在 C 里,你也许用过 `clock()` 计时:包含 `time.h`,前后各取一次 `clock()`,除以 `CLOCKS_PER_SEC`。这三个痛点:精度只有毫秒级、测的是 CPU 时间而非墙上时间、每次计时都要手写四行样板。C++ 的 `<chrono>` 库用 `steady_clock` 给出高精度单调时钟,一行 `duration` 模板化转换直接得到毫秒/微秒,还内置了类型检查——C 里"把秒和毫秒搞混"这类 bug 在 `chrono` 里根本写不出来。

Python 的 `time.perf_counter()` 与 C++ 的 `steady_clock` 思想相同:单调、高精度、不受系统改时间影响。本章 C++ 版与 Python 版的所有代码结构完全一致,只是计时函数不同——读完本章你会发现,算法分析是数学,语言只是载体。

本章路线:先建立记号体系(\$2.1-\$2.3),再学计算规则(\$2.4),然后在一个真实问题上亲手把 $O(n^3)$ 一路优化到 $O(n)$ (\$2.5),最后用计时实验验证理论(\$2.6-\$2.7)。

📦 前置知识自查

数学前置:指数与对数的基本性质(第1章)、求和记号 Σ 、递归四法则与递推式入门(第1章)。

C 语言前置:单层/嵌套 `for` 循环、函数定义与调用、数组下标遍历; `clock()` 计时(可选,会更好)。

依赖的前面章节:第1章绪论(递归思想、 $\log$ 与指数的运算性质)。

🚀 本章学习路线

1. **第一阶段(1 小时)**:读 §2.1–§2.3, 背下增长阶排序与常见函数值表
2. **第二阶段(1.5 小时)**:读 §2.4–§2.5, 亲手推导最大子序列和四种算法的复杂度, 并上机运行验证输出正确
3. **第三阶段(1.5 小时)**:读 §2.6–§2.7, 运行计时程序, 观察“n 翻倍、时间乘几”与理论预测是否吻合
4. **第四阶段(实验, 1 小时)**:完成章末实验: 四种算法的增长阶实测报告
5. **验收标准**:能随口说出“二分查找 $O(\log n)$ ”“快排平均 $O(n \log n)$ ”并解释理由

本章知识导图

- §2.1 为什么要分析算法:从“能跑”到“跑得完”(规模、计时工具、C 的 `clock()` 与 `chrono` 对照)
- §2.2 大 O 、 Ω 、 Θ 、小 o 定义、直觉、常见误用
- §2.3 增长阶排序与常见函数:七个等级、数值表
- §2.4 运行时间计算规则:顺序/循环/递归、求和记号
- §2.5 最大子序列和:四种算法, $O(n^3) \rightarrow O(n)$ 的递进
- §2.6 对数级复杂度的三种来源:二分查找、快速幂、欧几里得算法
- §2.7 实体验证:计时方法、n 翻倍实验、常数因子的意义

§2.1 为什么要分析算法

2.1.1 规模决定生死

同一个问题, 算法不同, 命运不同。假设处理一个元素耗时 1 微秒, 那么:

问题规模 n	$O(n \log n)$	$O(n^2)$	$O(n^3)$	$O(2^n)$
10^3	≈ 0.01 秒	1 秒	16.7 分钟	约 10^{286} 年
10^4	≈ 0.13 秒	100 秒	11.6 天	超出宇宙年龄
10^5	≈ 1.7 秒	2.8 小时	31.7 年	—
10^6	≈ 20 秒	11.6 天	3.2 万年	—

结论: 指数算法在任何现实规模上都不存在; 平方算法在十万级数据上就要等几小时。这就是为什么必须先学会“估计算法代价”——否则优化方向可能完全错误(去优化一个 $O(n)$ 的循环, 却对 $O(n^2)$ 的嵌套循环视而不见)。

2.1.2 从 C 的 `clock()` 到 C++ 的 `chrono`

C 的计时套路你也许写过:

C 的写法	C++ 的写法
<code>#include <time.h></code>	<code>#include <chrono></code>
<code>clock_t t0 = clock();</code>	<code>auto t0 = steady_clock::now();</code>
<code>double s = (double)(clock()-t0)/CLOCKS_PER_SEC;</code>	<code>auto ms = duration<double,milli>(steady_clock::now()-t0).count();</code>
精度毫秒级,测 CPU 时间	精度微秒/纳秒级,测墙上时间,类型安全

示例 2-1 C++ 计时框架(本章所有实验都用它)

```
#include <iostream>
#include <chrono>
using namespace std;
using Clock = chrono::steady_clock; // 单调时钟:不受系统改时间影响

int main() {
    auto t0 = Clock::now();
    // ---- 被测代码 ----
    long long s = 0;
    for (int i = 0; i < 1000000; ++i) s += i;
    // -----
    double ms = chrono::duration<double, milli>(Clock::now() - t0).count();
    cout << "耗时: " << ms << " ms, 结果 = " << s << endl;
    return 0;
}
```

★ 重点:计时不是分析的全部

计时是验证分析的工具,不是分析本身。同一程序换个机器、换个编译器、换个负载,时间可能差几倍——但“n 翻倍、时间约乘 4”这个比值规律几乎不变。复杂度分析研究的正是这个机器无关的规律。

§2.2 大 O、Ω、Θ、小 o

2.2.1 大 O:上界

定义(实用版): $T(n) = O(f(n))$ 表示存在常数 c 与 n_0 ,使得对所有 $n \geq n_0$,都有 $T(n) \leq c \cdot f(n)$ 。直白地说:当 n 足够大时, $T(n)$ 的增长速度不超过 $f(n)$ 的某个常数倍。例如 $T(n) = 3n^2 + 10n + 7$,取 $c = 4$ 、 n_0 足够大即可证明 $T(n) = O(n^2)$ ——因为 $10n + 7$ 相对 $3n^2$ 微不足道。

所以大 O 会“吞掉”低阶项与常数: $O(3n^2 + 10n + 7) = O(n^2)$, $O(2n) = O(n)$ 。这也是它的局限:两个同为 $O(n)$ 的算法可能实际相差 10 倍。

2.2.2 Ω 与 Θ:下界与紧界

$\Omega(f(n))$ 是对偶记号: $T(n) \geq c \cdot f(n)$ 对所有足够大的 n 成立,表示“增长不低于 $f(n)$ ”。而 $\Theta(f(n))$ 表示“恰好同阶”:既 $O(f(n))$ 又 $\Omega(f(n))$ 。工程上最常用的是大 O (上界保证“最坏不会更差”)与 Θ (精确刻画);当说“这个算法是 $\Theta(n \log n)$ ”时,意思是上下界都压紧了。

2.2.3 小 o 与三种常见误用

小 $o(f(n))$ 表示"增长速度严格低于 $f(n)$ "(大 O 允许相等,小 o 不允许): $n = o(n^2)$ 成立,但 $n \neq o(n)$ 。它常用于比较两个上界谁更"松"。

记号说明(本章约定,全书沿用)

正文中的 $O(n)$ 读作"大 O n "; n^2 表示 n 的平方; $\log_2 n$ 表示以 2 为底的对数,不写底数时 $\log n$ 默认以 2 为底(常数倍差异被大 O 吞掉,所以底数不重要,但推导过程要写清楚); $\ln$ 表示自然对数。求和写成 $\sum_{i=1}^n i = n(n+1)/2$ 。代码里一律使用语言语法(`%`、`>>`、`pow`),不使用数学记号。

误区 1:把常数当成增长阶

写 $O(2n)$ 、 $O(100n \log n)$ 、 $O(n+100)$ 都是错误的:大 O 定义明确吞掉常数,应写 $O(n)$ 、 $O(n \log n)$ 、 $O(n)$ 。"常数不重要"仅在分析增长阶时成立;实际工程选型时,两个 $O(n)$ 算法的常数差距可能决定生死。

误区 2:混淆最坏情形与记号本身

"快排是 $O(n^2)$ 的"——这句话必须补全:快排最坏情形是 $O(n^2)$,平均情形是 $O(n \log n)$ 。大 O 描述的是某个约定情形(最坏/平均/最好)下的增长阶,不是算法本身的属性。今后每章的复杂度速查表都会区分"平均"与"最坏"两列。

§2.3 增长阶排序与常见函数

2.3.1 七个等级

- $O(1)$ 常数:数组按下标访问、散列查找(平均)
- $O(\log n)$ 对数:二分查找—— n 翻倍只多一步
- $O(n)$ 线性:一趟遍历、线性搜索
- $O(n \log n)$ 线性对数:归并排序、快排(平均)、堆排序
- $O(n^2)$ 平方:二重循环、插入/冒泡排序、朴素矩阵乘法
- $O(n^3)$ 立方:三重循环、Floyd 全源最短路
- $O(2^n)$ 指数:穷举、朴素递归斐波那契——现实规模下不可用

2.3.2 数值表:让直觉落地

$f(n)$	$n=10$	$n=100$	$n=10^3$	$n=10^4$	$n=10^5$	$n=10^6$
$\log_2 n$	3.3	6.6	10	13	17	20
n	10	100	10^3	10^4	10^5	10^6
$n \log_2 n$	33	664	10^4	1.3×10^5	1.7×10^6	2×10^7
n^2	100	10^4	10^6	10^8	10^{10}	10^{12}
n^3	10^3	10^6	10^9	10^{12}	10^{15}	10^{18}
2^n	1024	约 10^{30}	约 10^{301}	约 10^{3010}	约 10^{30103}	约 10^{301030}

读这张表的要领:从 $n=10^3$ 到 10^6 , 规模扩大 1000 倍—— n 增长 1000 倍, n^2 增长 100 万倍, 2^n 则直接冲出宇宙。同时注意 $\log_2 n$ 一行:规模增长 1000 倍,它只从 10 长到 20。对数函数是“几乎不增长”的函数,凡是对数级的算法都值得信赖。

⚠ 误区 3:把 $2n$ 与 n^2 混为一谈

2^n 与 n^2 只差一个“位置”,命运完全不同: $n=64$ 时, $n^2=4096$, 而 $2^n \approx 1.8 \times 10^{19}$ 。判断口诀:底数是 n 的多项式,指数是 n 的才是指数级。 1.1^n 也是指数级(底数略大于 1 仍是灾难), n^{100} 再大也只是多项式。

§2.4 运行时间计算规则

2.4.1 三条基本规则

- **顺序相加**:两个语句依次执行,总时间取两者之和,大 O 取最大者: $O(n) + O(n^2) = O(n^2)$
- **嵌套相乘**:循环体内嵌套循环,总时间相乘
- **分支取最大**:if/else 取两个分支中较慢的一个

2.4.2 循环与求和记号

循环本质上是一个求和:第 i 次迭代的工作量 $w(i)$, 总工作量 $= \sum_i w(i)$ 。例:

示例 2-2 嵌套循环的求和推导

```
// 推导:内层循环执行  $n - i$  次
// 总次数  $= \sum_{i=0}^{n-1} (n - i) = n + (n-1) + \dots + 1 = n(n+1)/2$ 
// 所以是  $O(n^2)$ , 而不是“感觉上的  $n^2/2$ ”——常数  $1/2$  被大  $O$  吞掉
// 对照:C 的写法 for (i = 0; i < n; i++) 完全一致,结论不变
long long cnt = 0;
for (int i = 0; i < n; ++i)
    for (int j = i; j < n; ++j)
        ++cnt;
```

三个最常用的求和公式(本章及全书反复使用):

- $\sum_{i=1}^n i = n(n+1)/2$ (算术级数, $O(n^2)$)
- $\sum_{i=0}^n 2^i = 2^{n+1} - 1$ (几何级数, $O(2^n)$)
- $\sum_{i=1}^n 1/i \approx \ln n$ (调和级数, $O(\log n)$)

2.4.3 递归:递推式入门

递归代码的复杂度由**递推式**描述: $T(n) = 2T(n/2) + n$ 表示“把规模 n 的问题分成两个 $n/2$ 的子问题,再花 $O(n)$ 合并”。逐层展开:第 1 层代价 n , 第 2 层 $2 \cdot (n/2) = n$, 第 3 层 $4 \cdot (n/4) = n \dots$ 共 $\log_2 n$ 层, 每层 n , 总计 $O(n \log n)$ 。§2.5 的算法三正是这个递推式, §2.6 的三个对数算法则是 $T(n) = T(n/2) + O(1)$ 的特例——每层只进一个分支, 共 $\log_2 n$ 层, 每层常数代价。

★ 重点:两种最经典的递推式

$T(n) = T(n/2) + O(1) \rightarrow O(\log n)$ (二分查找); $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$ (归并/分治)。全书的对数与线性对数复杂度,九成来自这两个式子。

§2.5 最大子序列和:四种算法的递进

2.5.1 问题描述

给定整数序列 $a_1, a_2, \dots, a_n$ (可含负数), 求其**连续子序列的最大和**; 全为负数时约定答案为 0 (空子序列)。例如 $[-2, 11, -4, 13, -5, -2]$ 的最大子序列是 $[11, -4, 13]$, 和为 20。这是 Weiss 教材的经典例题: 同一个问题, 四种算法, 复杂度相差 100 万倍。

2.5.2 算法一: $O(n^3)$ 暴力枚举

最直接的思路: 枚举所有起点 i 与终点 j , 再累加 $a[i..j]$ 。三重循环, 外层 n 种起点, 中层 $n-i$ 种终点, 内层再花 $O(n)$ 求和, 总代价 $\sum \sum (j-i+1) \approx n^3/6$, 即 $O(n^3)$ 。

示例 2-3 算法一: 三重循环, $O(n^3)$

```
// 需要 #include <vector>
int maxSubSum1(const vector<int>& a) {
    int maxSum = 0;
    for (size_t i = 0; i < a.size(); ++i)    // 起点
        for (size_t j = i; j < a.size(); ++j) // 终点
        {
            int thisSum = 0;
            for (size_t k = i; k <= j; ++k)    // 累加 a[i..j]
                thisSum += a[k];
            if (thisSum > maxSum)
                maxSum = thisSum;
        }
    return maxSum;
}
```

2.5.3 算法二: $O(n^2)$ 复用中间和

观察: 固定起点 i , 当终点从 j 推进到 $j+1$ 时, 新和 = 旧和 + $a[j+1]$, 不需要重算整个区间。删掉内层循环, 复杂度降到 $\sum_{i=1}^n 1 = n(n+1)/2 = O(n^2)$ 。

示例 2-4 算法二: 两重循环, $O(n^2)$

```
int maxSubSum2(const vector<int>& a) {
    int maxSum = 0;
    for (size_t i = 0; i < a.size(); ++i) {
        int thisSum = 0;
        for (size_t j = i; j < a.size(); ++j) {
            thisSum += a[j];    // 增量更新, 不再重算
            if (thisSum > maxSum)
                maxSum = thisSum;
        }
    }
    return maxSum;
}
```

2.5.4 算法三: $O(n \log n)$ 分治

分治三步:把数组分成左右两半,递归求左半最大和与右半最大和,再求**跨中界的最大和**(从中界向左延伸的最大后缀和 + 向右延伸的最大前缀和),三者取最大。递推式正是 $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$ 。

示例 2-5 算法三:分治, $O(n \log n)$

```
// 需要 #include <algorithm> (std::max)
int maxSumRec(const vector<int>& a, int left, int right) {
    if (left == right) // 基准情形:单个元素
        return a[left] > 0 ? a[left] : 0;

    int center = (left + right) / 2;
    int maxLeft = maxSumRec(a, left, center); // 左半
    int maxRight = maxSumRec(a, center + 1, right); // 右半

    int maxLeftBorder = 0, leftBorder = 0; // 跨中界:向左延伸
    for (int i = center; i >= left; --i) {
        leftBorder += a[i];
        if (leftBorder > maxLeftBorder)
            maxLeftBorder = leftBorder;
    }
    int maxRightBorder = 0, rightBorder = 0; // 向右延伸
    for (int i = center + 1; i <= right; ++i) {
        rightBorder += a[i];
        if (rightBorder > maxRightBorder)
            maxRightBorder = rightBorder;
    }
    return max(max(maxLeft, maxRight),
               maxLeftBorder + maxRightBorder);
}

int maxSubSum3(const vector<int>& a) {
    return maxSumRec(a, 0, int(a.size()) - 1);
}
```

2.5.5 算法四: $O(n)$ 在线算法

关键观察: **和为负的前缀不可能是最优解的起点**——负前缀只会拖累后面。于是扫描一遍:累加当前和 thisSum,一旦 thisSum 变成负数就清零重来;途中记录最大值。

示例 2-6 算法四:在线算法, $O(n)$

```
int maxSubSum4(const vector<int>& a) {
    int maxSum = 0, thisSum = 0;
    for (int x : a) { // C++ 范围 for, 等价于按序遍历
        thisSum += x;
        if (thisSum > maxSum)
            maxSum = thisSum;
        else if (thisSum < 0)
            thisSum = 0; // 负前缀弃之,从下一个元素重新开始
    }
    return maxSum;
}
```

📄 例题 2-1 手工走一遍算法四

输入 $[-2, 11, -4, 13, -5, -2]$: thisSum 依次为 $-2 \rightarrow 0$ (清零)、11、7、20、15、13; maxSum 记录峰值为 20。答案 20, 与算法一一致。**注意:** 清零动作丢弃的是"负前缀"而不是负数本身—— $a[i]$ 单独为负但前缀仍正时不会清零。

★ 重点: 四种算法问题对照

算法	思想	复杂度	$n=10^6$ 时(1 微秒/步)
一	暴力枚举	$O(n^3)$	约 317 年
二	增量更新	$O(n^2)$	约 11.6 天
三	分治	$O(n \log n)$	约 20 秒
四	在线扫描	$O(n)$	约 1 秒

§2.6 对数级复杂度的三种来源

2.6.1 二分查找

在有序数组中查找 x : 每次把候选区间减半。区间长度 $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \rightarrow 1$, 共 $\log_2 n$ 步, 即递推式 $T(n) = T(n/2) + O(1)$ 。

示例 2-7 二分查找, $O(\log n)$

```
// 在升序数组 a 中查找 x, 返回下标; 找不到返回 -1
int binarySearch(const vector<int>& a, int x) {
    int low = 0, high = int(a.size()) - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2; // 防溢出的写法, 优于 (low+high)/2
        if (a[mid] < x) low = mid + 1;
        else if (a[mid] > x) high = mid - 1;
        else return mid;
    }
    return -1;
}
```

示例 2-8 标准库写法: binary_search 与 lower_bound

```
#include <algorithm>
vector<int> a = {1, 3, 5, 7, 9};
bool hit = binary_search(a.begin(), a.end(), 5); // true
// lower_bound: 第一个 >= x 的位置(不存在则返回 end())
auto it = lower_bound(a.begin(), a.end(), 6); // 指向 7
int pos = int(it - a.begin()); // 3
// upper_bound: 第一个 > x 的位置, 两者之差即"等于 x 的元素个数"
int cnt = int(upper_bound(a.begin(), a.end(), 5) - it);
```

C 的手写二分查找	C++ 的标准库
<pre>int bin(int a[], int n, int x)</pre> 手写约 12 行	<pre>binary_search / lower_bound / upper_bound</pre>
边界条件 (<code><=</code> 还是 <code><</code>) 极易写错成死循环	边界由标准库保证, 写错编译期就拦下
数组要单独传长度 <code>n</code>	迭代器区间 <code>[begin, end)</code> 自包含长度

手写 vs 标准库:学习时手写一遍理解"减半"骨架; 工程中直接用 `binary_search` (判断存在) 与 `lower_bound` (求位置) —— 后者返回"第一个不小于 `x` 的位置", 是插入有序序列时的正确姿势。Python 版对应 `bisect` 模块, 语义完全相同。

2.6.2 快速幂

计算 $a^n \bmod m$: 朴素做法乘 n 次是 $O(n)$; 利用 $a^n = (a^2)^{n/2}$ 每次把指数减半, $O(\log n)$ 。这是 C 里你可能用循环写过的算法, 与二分查找共享"减半"骨架。

示例 2-8 快速幂, $O(\log n)$

```
const long long MOD = 1000000007LL;
// 计算 (a^n) % MOD; n >= 0
long long powMod(long long a, long long n) {
    long long result = 1;
    while (n > 0) {
        if (n & 1)                // 当前二进制位为 1 则乘入
            result = result * a % MOD;
        a = a * a % MOD;          // a 平方: 对应指数减半
        n >>= 1;                  // n 右移一位 = n/2
    }
    return result;
}
```

2.6.3 欧几里得算法

求最大公因数 `gcd(a, b)`: 迭代执行 `(a, b) → (b, a % b)` 直到 `b = 0`。结论: 每两步至少把较小的数减半(证明略), 所以迭代次数 $O(\log n)$ 。

示例 2-9 欧几里得算法, $O(\log n)$

```
int gcd(int a, int b) {
    while (b != 0) {
        int t = a % b;
        a = b;
        b = t;
    }
    return a;
}
```

★ 重点: log 从哪来?

三个算法的共同骨架: 每一步把"问题规模"至少减半。二分查找减半区间, 快速幂减半指数, 欧几里得每两步减半数。凡是见到"减半""倍增"字样的算法, 先猜 $O(\log n)$ 再验证, 十有八九正确。

§2.7 实测验证:让理论落地

2.7.1 计时方法要点

- n 要足够大:太小则计时噪声淹没规律(§2.7.2 用 $n \geq 1000$)
- 重复取平均或取中位数,避免一次性的调度抖动
- 开优化编译(C++ 用 `/O2` 或 `-O2`),否则测得的是调试版速度
- **防止死代码消除**:编译器可能把"结果没被使用"的整段计算删掉——务必把结果累加进一个全局变量并输出

C 的计时要点	C++ 的计时要点
<code>clock()</code> 测 CPU 时间,受多线程/系统调度影响	<code>steady_clock</code> 测墙上时间,单调不回拨
除以 <code>CLOCKS_PER_SEC</code> 得到秒,容易忘记	<code>duration<double, milli></code> 直接出毫秒,类型安全
手动 <code>volatile</code> 防优化,易漏	同样要把结果累加输出;开 <code>/O2</code> 后验证

2.7.2 n 翻倍实验(C++ 实测参考值)

对随机数组运行四种算法,每次 n 翻倍,观察时间比值:

n	算法一	算法二	算法三	算法四
1000	412 ms	0.42 ms	0.05 ms	0.02 ms
2000	3310 ms	1.68 ms	0.11 ms	0.04 ms
4000	≈ 26 s(推算)	6.71 ms	0.24 ms	0.08 ms
8000	≈ 210 s(推算)	26.9 ms	0.51 ms	0.17 ms

读出规律: n 翻倍,算法一时间 $\times 8$ (吻合 n^3),算法二 $\times 4$ (吻合 n^2),算法三约 $\times 2.1$,算法四约 $\times 2.0$ (吻合 $n \log n$ 与 n)。**比值比绝对值可靠**——你的机器数值不同,但比值规律必然一致。Python 版做同样实验时,绝对时间约慢 30–100 倍(解释执行 + 每次运算都是对象操作),但**比值规律完全相同**:增长阶是语言无关的。

工程建议:常数因子的真实意义

大 O 相同不代表同样快:同为 $O(n \log n)$,快排的常数通常只有堆排的一半;同为 $O(n)$,遍历 `vector` 比遍历链表快数倍(缓存友好,第3章细讲)。工程选型时:先用大 O 淘汰指数/高次算法,再用常数因子与实测在候选者中择优。只背大 O 不看常数,和只看常数不懂大 O ,都是半桶水。

误区 4:小规模下拿 O 记号赌博

$n = 20$ 时, $O(n^2)$ 的插入排序往往比 $O(n \log n)$ 的快排更快——常数小。大 O 的结论只在" n 足够大"时成立(定义里的 n_0 正是为此)。工程上排序库普遍采用"大区间快排 + 小区间插入排序"的混合策略(第7章),就是这条原则的产物。

本章复杂度速查表

操作 / 算法	数据结构 / 算法	平均	最坏
按值查找	无序数组	$O(n)$	$O(n)$
二分查找	有序数组	$O(\log n)$	$O(\log n)$
求 a^n	朴素连乘	$O(n)$	$O(n)$
求 a^n	快速幂	$O(\log n)$	$O(\log n)$
最大公因数	欧几里得算法	$O(\log n)$	$O(\log n)$
最大子序列和	算法一(暴力)	$O(n^3)$	$O(n^3)$
最大子序列和	算法二(增量)	$O(n^2)$	$O(n^2)$
最大子序列和	算法三(分治)	$O(n \log n)$	$O(n \log n)$
最大子序列和	算法四(在线)	$O(n)$	$O(n)$
循环计数	单层循环	$O(n)$	$O(n)$
循环计数	双重循环	$O(n^2)$	$O(n^2)$
递归	$T(n) = T(n/2) + O(1)$	$O(\log n)$	$O(\log n)$
递归	$T(n) = 2T(n/2) + O(n)$	$O(n \log n)$	$O(n \log n)$
增长阶	常数 / 对数 / 线性 / 线性对数 / 平方 / 立方 / 指数	见 §2.3.1 排序	
增长阶排序	常数 $< \log n < n < n \log n < n^2 < n^3 < 2^n$	全书唯一需要背诵的排序	

最小必会清单

- 能独立说出大 O 、 Ω 、 Θ 、小 o 的定义与区别,并各举一例
- 能背诵增长阶排序,并给每个等级配一个真实算法例子
- 能独立推导单层循环、嵌套循环、if/else 的复杂度
- 能独立写出四种最大子序列和算法,并说出各自复杂度与推导
- 能独立写出二分查找、快速幂、gcd 的代码,并解释为什么是 $O(\log n)$
- 能写出 $T(n) = T(n/2) + O(1)$ 与 $T(n) = 2T(n/2) + O(n)$ 的解
- 能解释"最坏情形"与"平均情形"的区别,并正确表述快排的复杂度
- 能独立用计时工具做 n 翻倍实验,判断实验比值与理论增长阶是否吻合

自测题

Q 1. $O(2n)$ 与 $O(n)$ 是否相等?为什么?

✅ 参考答案

相等。大 O 定义允许任意常数倍:取 $c = 2$ 即有 $2n \leq 2 \cdot n, 2n = O(n)$ 。反过来 $n \leq 1 \cdot 2n$, 所以两者互为对方的 O , 增长阶相同。写 $O(2n)$ 是不规范的。

Q 2. 某算法运行时间 $T(n) = 3n^2 + 10n + 7$, 它的 O 、 Ω 、 Θ 分别可以写什么?

✓ 参考答案

$O(n^2)$ (也允许 $O(n^3)$ 等更松的上界,但不规范); $\Omega(n^2)$ (也允许 $\Omega(n)$ 等更松的下界); $\Theta(n^2)$ 是唯一"紧"的写法。

Q 3. 为什么说算法三(分治)比算法二(增量)快?写出算法三的递推式并解释每层代价。

✓ 参考答案

$T(n) = 2T(n/2) + n$:把数组分为两半递归($2T(n/2)$),再花 $O(n)$ 求跨中界最大和。展开后第 k 层有 2^k 个规模 $n/2^k$ 的子问题,每层总代价都是 n ,共 $\log_2 n$ 层,总计 $n \log_2 n$ 。 $O(n \log n)$ 在 $n = 10^6$ 时约 2×10^7 步,而 $O(n^2)$ 是 10^{12} 步,差 5 万倍。

Q 4. 二分查找每次把区间减半,为什么复杂度是 $O(\log n)$ 而不是 $O(n/2)$?

✓ 参考答案

区间长度依次为 $n, n/2, n/4, \dots, 1$,问"减半多少次到 1"——这是对数: $2^k = n$,即 $k = \log_2 n$ 次。 $n/2$ 是"一次减半后的长度",不是步数;把长度当步数是常见错误。 10^6 个元素只需约 20 次比较。

Q 5. $n = 1000$ 时, $O(n^2)$ 与 $O(n \log n)$ 的步数约相差多少倍? $n = 10^6$ 呢?

✓ 参考答案

$n = 1000: 10^6 / (1000 \times 10) = 100$ 倍; $n = 10^6: 10^{12} / (10^6 \times 20) = 5 \times 10^4$ 倍。规模越大差距越大——这就是为什么"复杂度差的算法靠硬件换代救不回来"。

Q 6. Python 程序比 C++ 慢约 50 倍,那么 $O(n)$ 的 Python 算法与 $O(n \log n)$ 的 C++ 算法,数据规模多大时 Python 反而更快?

✓ 参考答案

比较 $50n$ 与 $n \log_2 n$:当 $\log_2 n > 50$,即 $n > 2^{50} \approx 10^{15}$ 时 Python 更快——现实规模基本达不到,所以"语言换效率"在增长阶面前通常无效。反过来: $O(n^2)$ 的 C++ 在大 n 下照样被 $O(n \log n)$ 的 Python 反超。

章末实验题:四种算法的增长阶实测报告

实验 2:验证最大子序列和四种算法的增长阶

实验目标:实现四种最大子序列和算法,对倍增规模计时,用实测比值验证 $O(n^3)/O(n^2)/O(n \log n)/O(n)$ 的增长阶预测。

实验要求:

- 任务 1:实现 maxSubSum1..4 四个函数(知识点:\$2.5 四种算法)
- 任务 2:用 `steady_clock` 计时,输出 $n = 1000/2000/4000/8000$ 的时间表(知识点:\$2.1 计时工具)
- 任务 3:算法一在 $n \geq 4000$ 时跳过并标注"(推算)",避免无谓等待(知识点:\$2.3 指数/立方增长不可用)
- 任务 4:输出" n 翻倍时各算法时间比值",与理论值($\times 8/\times 4/\times 2/\times 2$)对比并写一段结论(知识点:\$2.7 实测验证)

实验环境:C++17,MSVC 或 g++。编译:`cl /EHsc /std:c++17 lab2.cpp` 或 `g++ -std=c++17 lab2.cpp -o lab2`;运行 `lab2`。

第一步 写四个算法函数,用 \$2.5 的样例数组 `[-2, 11, -4, 13, -5, -2]` 验证四个函数都输出 20,再写计时主程序

第二步 随机数用 `mt19937 + uniform_int_distribution<int>(-1000, 1000)`,固定种子 12345 保证可复现

第三步 时函数接收函数指针,复用同一份计时代码;注意把返回值累加并最终输出,防止编译器死代码消除

第四步 比值分析:对算法二,若 n 翻倍时间约 $\times 4$,则吻合 n^2 ;对算法三、四约 $\times 2$;算法一只测两个规模并说明 $\times 8$ 趋势

✔ 参考答案(完整可运行程序,约 90 行,分两段)

```

// lab2.cpp — 最大子序列和:四种算法的增长阶实测(第一段:算法)
#include <iostream>
#include <vector>
#include <random>
#include <chrono>
#include <algorithm>
using namespace std;
using Clock = chrono::steady_clock;

// ----- 四种算法($2.5) -----
int maxSubSum1(const vector<int>& a) {           //  $O(n^3)$ 
    int maxSum = 0;
    for (size_t i = 0; i < a.size(); ++i)
        for (size_t j = i; j < a.size(); ++j) {
            int thisSum = 0;
            for (size_t k = i; k <= j; ++k) thisSum += a[k];
            if (thisSum > maxSum) maxSum = thisSum;
        }
    return maxSum;
}

int maxSubSum2(const vector<int>& a) {           //  $O(n^2)$ 
    int maxSum = 0;
    for (size_t i = 0; i < a.size(); ++i) {
        int thisSum = 0;
        for (size_t j = i; j < a.size(); ++j) {
            thisSum += a[j];
            if (thisSum > maxSum) maxSum = thisSum;
        }
    }
    return maxSum;
}

int maxSumRec(const vector<int>& a, int left, int right) { //  $O(n \log n)$ 
    if (left == right) return a[left] > 0 ? a[left] : 0;
    int center = (left + right) / 2;
    int maxLeft = maxSumRec(a, left, center);
    int maxRight = maxSumRec(a, center + 1, right);
    int maxLeftBorder = 0, leftBorder = 0;
    for (int i = center; i >= left; --i) {
        leftBorder += a[i];
        if (leftBorder > maxLeftBorder) maxLeftBorder = leftBorder;
    }
    int maxRightBorder = 0, rightBorder = 0;
    for (int i = center + 1; i <= right; ++i) {
        rightBorder += a[i];
        if (rightBorder > maxRightBorder) maxRightBorder = rightBorder;
    }
    return max(max(maxLeft, maxRight), maxLeftBorder + maxRightBorder);
}

int maxSubSum3(const vector<int>& a) {
    return maxSumRec(a, 0, int(a.size()) - 1);
}

```

```
int maxSubSum4(const vector<int>& a) {           // O(n) 在线算法
    int maxSum = 0, thisSum = 0;
    for (int x : a) {
        thisSum += x;
        if (thisSum > maxSum) maxSum = thisSum;
        else if (thisSum < 0) thisSum = 0;
    }
    return maxSum;
}
```

lab2.cpp(第二段:计时与主程序)

```
// ----- 计时($2.1) -----
using Func = int (*)(const vector<int>&);
double msTime(Func f, const vector<int>& a, long long& guard) {
    auto t0 = Clock::now();
    guard += f(a);           // 返回值累加,防止死代码消除
    return chrono::duration<double, milli>(Clock::now() - t0).count();
}

int main() {
    // 正确性验证($2.5 样例)
    vector<int> demo = {-2, 11, -4, 13, -5, -2};
    cout << "验证: " << maxSubSum1(demo) << " "
        << maxSubSum2(demo) << " " << maxSubSum3(demo) << " "
        << maxSubSum4(demo) << " (应全为 20)" << endl;

    mt19937 rng(12345);
    uniform_int_distribution<int> dist(-1000, 1000);
    long long guard = 0;

    cout << "n\t算法一\t算法二\t算法三\t算法四\n";
    for (int n : {1000, 2000, 4000, 8000}) {
        vector<int> a(n);
        for (int& x : a) x = dist(rng);
        cout << n;
        if (n <= 2000)
            cout << "\t" << msTime(maxSubSum1, a, guard);
        else
            cout << "\t--(推算)"; // O(n^3) 在 4000 时约 26 秒,不实测
        cout << "\t" << msTime(maxSubSum2, a, guard)
            << "\t" << msTime(maxSubSum3, a, guard)
            << "\t" << msTime(maxSubSum4, a, guard) << endl;
    }
    cout << "(防优化累加值: " << guard % 1000 << ")" << endl;
    cout << "结论: n 翻倍时,算法一 x8(吻合 n^3),算法二 x4(吻合 n^2),"
        << "算法三/四约 x2(吻合 n log n 与 n)。" << endl;
    return 0;
}
```

示例输出(机器不同数值不同,比值规律一致):

验证: 20 20 20 20 (应全为 20)

n	算法一	算法二	算法三	算法四
1000	412.3	0.42	0.05	0.02
2000	3310.1	1.68	0.11	0.04
4000	--(推算) 6.71		0.24	0.08
8000	--(推算) 26.9		0.51	0.17

(防优化累加值: 123)

结论: n 翻倍时, 算法一 $\times 8$ (吻合 n^3), 算法二 $\times 4$ (吻合 n^2), 算法三/四约 $\times 2$ (吻合 $n \log n$ 与 n)。

设计说明:① 四个算法共用 `Func` 函数指针类型, 计时代码只写一份——这正是第7章"比较器"思想的雏形, 也与 C 的 `qsort` 函数指针同源; ② `guard` 变量累加所有返回值并输出, 防止编译器把整个调用优化掉; ③ 算法一在 $n \geq 4000$ 时按 §2.3 的数值表推算($\times 8$ 每翻倍), 避免把几分钟浪费在早已被理论淘汰的算法上; ④ 种子固定为 12345, 任何机器复现本实验都得到相同的数组。

下一章预告: 第3章 表、栈和队列——三种最基础的数据结构及其数组/链式实现。第3章起, 每章末尾的复杂度速查表将直接使用本章的记号; 最大子序列和的分治骨架也将在第7章排序中再次登场。